Internship Summary Report

Yanjia Kan

I. Overview

From May to June 2025, I worked as a Summer Intern at Google, focusing on artificial intelligence algorithm development and model optimization. During this period, I independently completed a series of deep learning projects tailored to real-world business needs, including:

- Framework migration and performance benchmarking for image classification models
- Prototype development and evaluation of a multimodal image-text retrieval system
- Construction and deployment of an NLP intent recognition service based on Hugging Face Transformers

Throughout the internship, I closely adhered to Google's AI development workflows and quality assurance standards. I independently carried out tasks involving model architecture design, data preprocessing, algorithm development, performance evaluation, and technical documentation, gaining substantial hands-on engineering experience. This also deepened my understanding and proficiency in TensorFlow/Keras, Hugging Face Transformers, Vertex AI, and the TensorFlow Extended (TFX) toolchain.

Each of the three main projects targeted distinct technical domains and business applications. Through this experience, I not only enhanced my practical capabilities in applying AI technologies to Google's real-world use cases but also cultivated efficient communication and proactive problem-solving skills in a remote work setting—laying a solid foundation for my future career development.

2. Technical Achievements

2.1 Project 1: Model Migration from PyTorch to TensorFlow and Framework Performance Benchmarking

Project Background:

In Google's internal deep learning projects, the choice of framework plays a critical role in development efficiency, system performance, and large-scale deployment. To gain a deeper understanding of the performance differences between PyTorch and TensorFlow—two of the most widely used deep learning frameworks—in Google's cloud production environment, the company initiated a comprehensive model migration and performance evaluation task. The goal was to generate data-driven insights to inform future decisions regarding the team's technology stack.

In this project, I independently implemented the migration of a pre-trained ResNet50 model from PyTorch to the TensorFlow/Keras framework. I then conducted a thorough quantitative analysis comparing the two frameworks in terms of training efficiency, inference latency, memory usage, and development convenience. This end-to-end process—from data preprocessing and model construction to fine-tuning, benchmarking, and technical documentation—resulted in a well-structured evaluation report that provides a solid foundation for future framework selection in Google AI projects.

Project Objectives:

- Migrate a pre-trained ResNet50 model from PyTorch to TensorFlow with equivalent functionality.
- Perform comparative benchmarking across key performance metrics: training speed, inference throughput, accuracy, and memory usage.
- Evaluate framework compatibility and efficiency within the Google Vertex AI cloud environment to support framework selection in future Google AI initiatives.

Implementation Process:

- **Defined Experimental Environment:**
  Set up model training and evaluation workflows on Google Cloud's Vertex AI platform.
- **Standardized Experimental Design:**
  Utilized the public Oxford Flowers 102 dataset, with an 80/20 split for training and validation. Standardized preprocessing included resizing (224×224), normalization, and data augmentation across both frameworks.
- **Model Migration and Optimization:**
  Employed the ResNet50 architecture with ImageNet pre-trained weights. Added a custom

classification head (GlobalAveragePooling2D → Dense → Dropout). The backbone network was frozen, and only the classification layers were fine-tuned.

- **Unified Hyperparameters:**
  Used consistent training settings across frameworks: batch size = 32, learning rate = 1e-3, epochs = 10, and Adam optimizer, to ensure fairness and reproducibility in performance comparison.
- **Performance Metrics Collection**:
  Carefully recorded training time, memory usage, inference latency, model accuracy, and loss. These metrics were compiled into comparative graphs for intuitive analysis.

Experimental Results

Through comprehensive experimentation and comparative analysis, TensorFlow demonstrated clear advantages over PyTorch across several key performance metrics, including training efficiency, inference latency, and validation accuracy. The findings are as follows:

- **Validation Accuracy**:
  TensorFlow achieved a validation accuracy of 0.64, substantially higher than PyTorch's 0.29.
- **Training Efficiency**:
  TensorFlow outperformed PyTorch by approximately 40% in training speed, largely due to the optimized data pipeline provided by the tf.data API.
- **Inference Latency**:
  The TensorFlow model exhibited roughly 80% lower inference latency compared to PyTorch, making it more suitable for production-level deployment.
- **Memory Usage**:
  PyTorch consumed less memory overall, making it more appropriate for resource-constrained research environments.

Overall, TensorFlow offers a more compelling framework for enterprise-grade deployment scenarios, with significant advantages in scalability and ease of deployment. The *tf.data* pipeline, in particular, contributed to marked improvements in disk I/O throughput, further enhancing data-loading efficiency in large-scale training workflows.

2.2 Project 2: Development of a Multimodal Image-Text Retrieval Prototype Based on Open Images Localized Narratives

Project Background

Multimodal retrieval has broad applications in intelligent human-computer interaction, precise content search,

and annotation assistance. To explore the multimodal potential of Google's open-source *Open Images Dataset V7*, the company initiated the development of a prototype system for image-text retrieval. The goal was to allow users to query with natural language and retrieve not only relevant images but also their corresponding localized regions.

In this project, I independently led the end-to-end development of the prototype, including data preprocessing and cleaning, multimodal model selection and application, system integration, and performance evaluation. The outcome was a functional, modular multimodal retrieval system that demonstrated effective image-text alignment and region-level retrieval capabilities. This system provides a foundational technical solution for future enhancements and productization.

Project Objectives

- Construct a multimodal image-text retrieval system based on the *Localized Narratives* subset of Open Images V7.
- Enable region-level retrieval of relevant image areas based on free-form natural language queries.
- Investigate and select suitable multimodal embedding models for local prototyping and system deployment.
- Evaluate the system's retrieval performance in real-world scenarios, with particular emphasis on localized region accuracy.

Implementation Process

- **Data Sampling and Preprocessing**:
  From the complete *Open Images V7 Localized Narratives* dataset, a total of 50,980 annotated samples from the first training batch were selected. This involved downloading and cleaning 19,354 images, including spatial-temporal filtering of trajectories and image validity checks. After data re-splitting, the final dataset was divided into training, validation, and test sets in a 75%:6%:19% ratio.
- **Multimodal Model Selection**:
  A thorough review was conducted on state-of-the-art multimodal embedding models such as CLIP and ALIGN. Considering local experimental resource constraints and the lightweight requirements of the prototype stage, OpenAI's CLIP model was ultimately selected to achieve efficient and lightweight image-text semantic alignment.
- **System Construction and Feature Extraction**:
  Textual features were extracted using the Sentence-BERT model, followed by a linear projection layer and L2 normalization to ensure dimensional consistency. Image features were directly extracted using

the pretrained CLIP model. An offline retrieval database was built using Annoy indexing to support fast online retrieval from captions to corresponding image regions.

- **Performance Evaluation**:
  A comprehensive evaluation framework was established, incorporating metrics such as *Precision@K*, *Recall@K*, Mean Average Precision (MAP), Normalized Discounted Cumulative Gain (NDCG), and *PointCoverage@K*. Particular emphasis was placed on *PointCoverage@K*, a metric specifically designed to assess the accuracy and coverage of localized region retrieval.

Experimental Results

The prototype system successfully enabled effective retrieval and localization of relevant image regions based on natural language queries, achieving the following key outcomes:

- In global cross-image retrieval tasks, the system demonstrated the ability to identify and localize semantically relevant regions, with especially strong performance on frequent concepts such as object-category and color combinations.
- In local image retrieval tasks (i.e., identifying target regions within a single image), the system achieved relatively accurate localization of target semantic areas. However, the trajectory point coverage (*PointCoverage@5*) reached only 35%, indicating limitations in detecting sparse or spatially dispersed trajectories.

Overall, the prototype system provides a preliminary but effective validation of the CLIP model's feasibility and potential in multimodal retrieval applications. The results offer valuable data-driven insights for further system refinement and optimization.

2.3 Project 3: Development and Evaluation of an NLP Intent Recognition Module Based on HuggingFace Transformers

Project Background

In various user-facing services at Google, natural language understanding (NLU) plays a vital role in enhancing user experience and improving the efficiency of automated systems. To more accurately identify user intent and enable seamless automation, the company aimed to develop a high-performance, scalable intent recognition module that could be easily integrated into existing service infrastructures.

In this project, I independently developed a complete NLP module based on the Hugging Face Transformers library with a TensorFlow backend. This included data preprocessing, model fine-tuning, API encapsulation, and performance evaluation. The result is a well-structured, production-ready service component designed to meet the requirements of deployment within the Google ecosystem.

Project Objectives

- Design and build a high-accuracy, high-efficiency intent recognition model with scalability for future business scenarios.
- Train and evaluate the model on a publicly available benchmark dataset (CLINC150), providing comprehensive quantitative performance metrics.
- Utilize the Hugging Face Transformers library with TensorFlow as the backend, implementing the module in compliance with Google AI service development standards.
- Deliver a preliminary API encapsulation of the inference service to enable rapid validation and demonstration.
- Explore strategies for inference optimization and privacy protection to meet the demands of production-level deployment.

Implementation Process

- **Data Processing and Privacy Protection**:
  The CLINC150 dataset was used as the benchmark. Preprocessing steps included tokenization, padding, and class balancing. Preliminary research was conducted on text anonymization (removal of Personally Identifiable Information, PII) and differential privacy techniques (e.g., DP-SGD), laying a technical foundation for user data protection in future production scenarios.
- **Model Selection and Architecture Design**:
  Comparative analysis was performed among *DistilBERT*, *ALBERT*, *RoBERT*, and traditional SVM models. The final choice was the TensorFlow implementation of BERT-base (*TFBertForSequenceClassification*), selected for its robust pretraining performance. The architecture was fine-tuned with scalability in mind, including future options for model distillation and deployment optimization.
- **Model Training and Evaluation**:
  Fine-tuning was conducted using Hugging Face Transformers with a TensorFlow backend, optimized with the AdamW optimizer and warm-up strategies. The evaluation focused on overall model performance, classification accuracy across varying text lengths, and inference efficiency. Metrics included Accuracy, Precision, Recall, F1-score, and latency per inference. Performance across short (≤5 words) and long (>12 words) text segments was also analyzed.
- **Module Packaging and Deployment Strategy**:
  A prototype inference API was developed using FastAPI, along with a standardized API design draft. Initial strategies for large-scale deployment were proposed, including model distillation, ONNX Runtime acceleration, and dynamic batching, to ensure readiness for Google-scale production

environments.

- **Bias Detection and Mitigation Strategies**:
  A preliminary analysis identified potential bias issues in the CLINC150 dataset. Mitigation strategies such as data augmentation, reweighting, and adversarial debiasing were explored as potential solutions for improving model fairness.

Project Outcomes

This project successfully delivered an efficient and accurate NLP intent recognition service module, with the following quantitative results:

- **Overall Performance**: Achieved an accuracy of 95.9% and an F1-score of 0.959 on the test set.
- **Text-Length Performance**: Reached 97% accuracy on short texts (≤5 words) and 95.3% accuracy on longer texts (>12 words), meeting practical application requirements.
- **Inference Efficiency**: Average inference latency was approximately 102 ms per sample, demonstrating initial suitability for production deployment.
- **Privacy-Preserving Techniques**: Identified appropriate use cases for PII removal and differential privacy methods, providing a technical reference for future user data protection efforts.
- **MLOps and TFX Planning**: Conducted initial exploration of integrating TensorFlow Extended (TFX) for data validation, continuous training, and deployment monitoring in production workflows.
- **Standardized API Documentation**: Developed a detailed API design draft aligned with Google's API design standards to support future integration and scalability.

In summary, this NLP module effectively fulfills the business objective of enhancing user understanding capabilities in Google AI services. It provides a solid foundation for further improvements in inference performance and bias mitigation, with the potential to support large-scale deployment in production-grade environments.

3. Challenges and Solutions

3.1 Project 1

Challenge 1: Framework Interface Discrepancies Leading to Accuracy Deviation

- **Issue**: After the initial migration, the TensorFlow model achieved only 29% accuracy, significantly lower than the original PyTorch version (64%).
- **Solution**: Standardized the data preprocessing pipeline and explicitly specified the dimension order in

tf.transpose. Replaced manual normalization with a customized ImageNet standardization layer in TensorFlow, and added a consistency check on channel ordering in the validation set to ensure alignment across frameworks.

Challenge 2: Environment Setup Failures

- **Issue**: Early attempts to set up the environment locally encountered compatibility issues.
- **Solution**: Successfully resolved the issue by switching to the Vertex AI cloud platform, which enabled rapid and automated environment provisioning.

3.2 Project 2

Challenge 1: Dataset Size and Hardware Limitations

- **Issue**: The full Open Images dataset is excessively large, making it costly to download and store, and infeasible to experiment with under limited local computing resources.
- **Solution**: Selected the first training batch (approximately 50,000 records) as a representative subset. After data cleaning and partitioning, the resulting dataset was scaled appropriately for local experimentation.

Challenge 2: Selection of Multimodal Embedding Model

- **Issue**: There is a wide range of multimodal models, each with varying resource requirements, making it difficult to select an optimal model.
- **Solution**: Based on literature review and preliminary experiments, the CLIP model was chosen for its strong pretrained performance, moderate resource demands, and ease of deployment—effectively resolving the resource bottleneck.

Challenge 3: Dimensional Mismatch Between Text and Image Features

- **Issue**: Features extracted from Sentence-BERT (384-dimensional) did not match the 512-dimensional CLIP image embeddings, which impeded similarity computation.
- **Solution**: Introduced a linear projection layer to map text features from 384 to 512 dimensions, followed by L2 normalization to ensure consistent feature representation.

Challenge 4: Retrieval Speed and Efficiency

- **Issue**: Online retrieval was initially too slow to meet real-time application requirements.
- **Solution**: Implemented a prebuilt Annoy index database to accelerate retrieval, significantly improving performance and enabling fast, real-time query handling.

3.3 Project 3

Challenge 1: Class Imbalance in Intent Categories

- **Issue**: The original CLINC150 dataset exhibited severe class imbalance, particularly with intent category 42, which could significantly degrade model training performance.
- **Solution**: During preprocessing, the severely imbalanced category was removed. The dataset was rebalanced, which improved the model's overall generalization capability.

Challenge 2: Model Overfitting

- **Issue**: Beginning from epoch 5, validation loss started to increase, indicating overfitting.
- **Solution**: Employed an early stopping strategy that terminated training after 5 epochs without improvement. This effectively prevented overfitting and preserved the model's generalization performance.

Challenge 3: API Standardization

- **Issue**: The initial API implementation did not conform to Google's standard design guidelines, which could hinder future deployment and scalability.
- **Solution**: Drafted a formal API documentation based on Google's public [API Design Guide], clearly defining endpoints, protocols, and error handling specifications. This enhanced the API's standardization and future extensibility.

4. Reflections and Future Outlook

Through the hands-on experience gained from multiple projects during this internship, I have developed a clearer understanding of future directions for technical optimization, system deployment, and engineering practice. To address the remaining technical challenges encountered in the projects, future work can focus on the following areas:

4.1. Continuous Optimization of MLOps Pipeline Development

- This internship included preliminary exploration of Google Vertex AI and TensorFlow Extended (TFX), which demonstrated significant potential for data processing, model training, and automated deployment. However, due to limitations of the Windows development environment, their full capabilities were not extensively explored.
- Future plans include migrating to a Linux-based environment to fully leverage TFX and the Vertex AI platform, and to establish a robust, end-to-end MLOps pipeline covering data validation, model

training and evaluation, deployment, and monitoring.

- The performance of the migrated TensorFlow models will be continuously monitored in production. Regular retraining and automated updates will be implemented, alongside experimentation with model version control and quality assurance strategies.

4.2. Building Standardized Data Processing Pipelines

- The project has already demonstrated the efficiency and consistency of tf.data pipelines for image data loading. However, no unified internal standard or best practice has been established.
- Future work will involve learning from best practices in tf.data optimization and aligning with the company's standardized data processing specifications to further improve model training efficiency and overall development productivity.

4.3. Improving Localized Image-Text Retrieval Precision

- In the image-text retrieval project, the model showed limited ability to detect dispersed target regions, which impacted the practical utility of localized retrieval.
- Future improvements will focus on fine-grained model tuning and enhanced multimodal attention mechanisms to increase region sensitivity and coverage, thereby improving the PointCoverage metric and user experience.

4.4. Addressing Input Length Limitations in NLP Models

- The current BERT-based intent classification model uses a fixed input length of 32 tokens, which may result in information loss for longer texts, limiting classification accuracy.
- Future work will involve increasing the maximum input length (e.g., to 64 tokens) and applying a sliding window truncation method to improve performance on longer text inputs, thereby accommodating more complex application scenarios.

4.5. Enhancing Inference Efficiency in NLP Models

- The current NLP intent recognition module has an average inference latency of ~102 ms/sample, which is sufficient for prototype demonstration but may fall short in large-scale, production-grade deployments at Google scale.
- Future optimization will explore model distillation techniques (e.g., DistilBERT, TinyBERT) and inference

acceleration engines (e.g., ONNX Runtime, TensorRT, XLA compilation) to significantly reduce latency and improve throughput and deployment cost-efficiency.

By systematically addressing the challenges identified across the projects and implementing the proposed optimization strategies, future efforts can further consolidate and extend the project outcomes. These improvements will help integrate the developed systems into real-world production environments, enhance engineering efficiency and model performance, and ultimately contribute to the realization of industrial-grade AI service deployment at Google scale.

5. Conclusion

5.1 Reflections on Remote Independent Work

During this internship, I independently completed multiple AI projects under real-world business scenarios in a fully remote setting. This experience provided deep insights into both the opportunities and challenges of remote independent work. The greatest advantage of remote work lies in its flexibility and autonomy, allowing for maximum personal initiative. Throughout the projects, I honed my ability to learn rapidly and solve problems independently. At the same time, remote work placed higher demands on communication efficiency and project management skills.

First, in the absence of face-to-face interaction, clear and efficient communication became essential. During the internship, I took the initiative to regularly report project progress and encountered issues to my mentor, clarify task requirements, and align deliverables with expectations.

Second, working independently required detailed planning regarding task goals, technical paths, and project timelines. Based on the project requirements assigned by supervisors, I formulated a clear project schedule and milestone-based objectives. Despite occasional external constraints, I ensured high-quality completion of all assigned tasks.

Lastly, well-structured documentation is a cornerstone of remote collaboration. Throughout the internship, I strictly followed documentation standards, diligently recorded project methodologies and technical details, and proactively documented key findings and lessons learned. This not only preserved institutional knowledge but also ensured smooth information flow within the team.

Overall, this remote working experience significantly enhanced my self-motivation, communication, and project management capabilities. It also deepened my understanding of the critical role that self-directed learning and continuous exploration play in the long-term development of technical professionals.

5.2 Insights for Future Career Development

This internship provided valuable opportunities to apply cutting-edge AI technologies in real enterprise

projects, helping me clarify my future career direction and better understand the skills and competencies required for professional growth.

Firstly, the internship highlighted the importance of enterprise-level MLOps in modern AI applications. This offered me practical experience and foundational knowledge. I plan to further explore core MLOps technologies to enhance my competitiveness in building scalable, production-ready AI systems.

Secondly, improving my software engineering and deployment capabilities has become a key focus for my future development. During the internship, I gained hands-on experience in API development, standardized design, and performance optimization. I realized that the successful deployment of AI models critically depends on solid engineering practices. To address current skill gaps, I will focus on improving implementation and performance tuning skills, and continue learning best practices in model distillation, inference acceleration, cloud deployment, and API design standards.

Lastly, the internship exposed me to cross-domain multimodal technologies, which revealed the technical complexity involved in solving real-world problems. Furthermore, industrial AI applications often require attention to data privacy, model bias, and AI ethics. In future career development, beyond mastering core technologies, I intend to enhance my awareness and practice around these real-world challenges.

In summary, this internship not only solidified my technical foundation but also helped me define a clear path and vision for future career development. I will continue to pursue in-depth learning, stay engaged with emerging technologies, and consistently improve my comprehensive capabilities—striving toward becoming an AI professional with strong engineering skills and a broad technical perspective.